

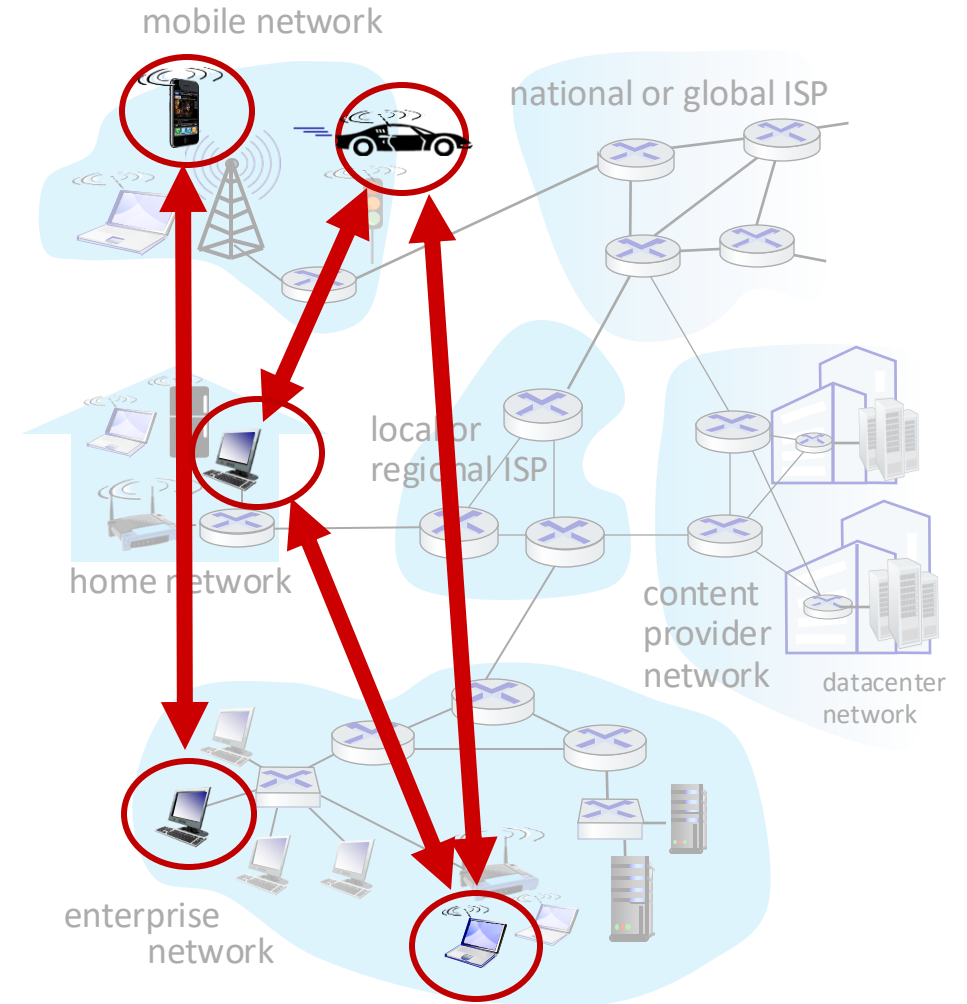
Application Layer: Overview

- Principles of network applications
- Web and HTTP
- E-mail, SMTP, IMAP
- The Domain Name System DNS
- P2P applications
 - video streaming and content distribution networks
 - socket programming with UDP and TCP

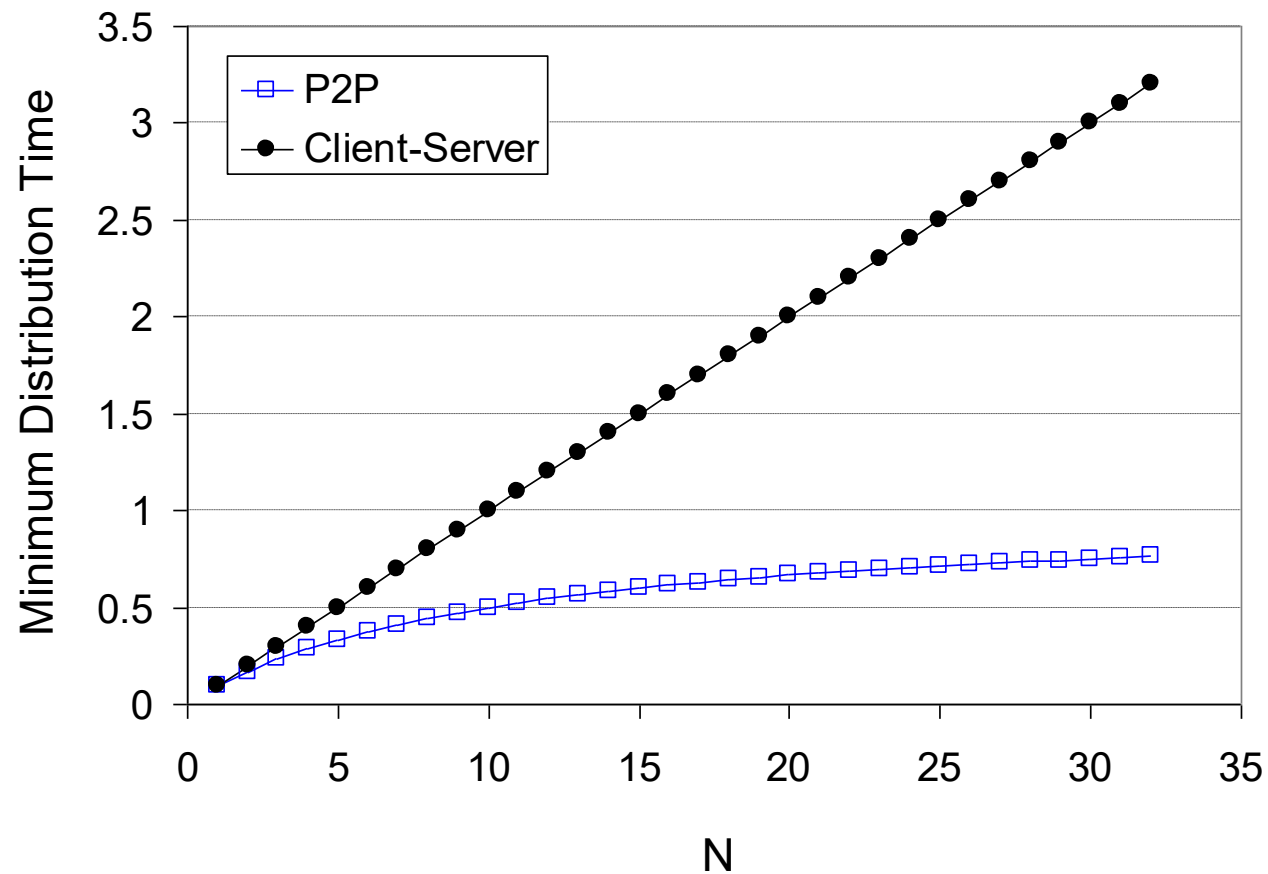


Peer-to-peer (P2P) architecture

- *no* always-on server
- arbitrary end systems directly communicate
- peers request service from other peers, provide service in return to other peers
 - *self scalability* – new peers bring new service capacity, and new service demands
- peers are intermittently connected and change IP addresses
 - complex management
- examples: P2P file sharing (BitTorrent), streaming (KanKan), VoIP (Skype)



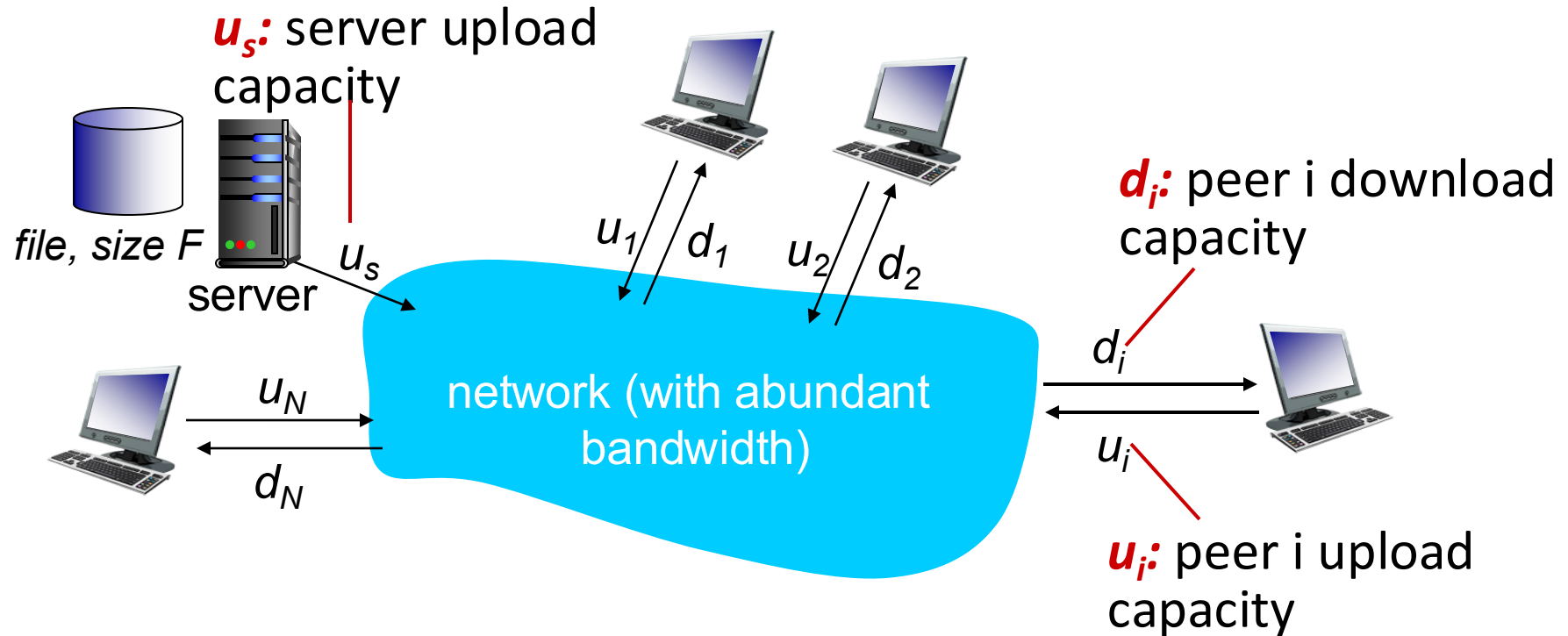
Klarer vi å si noe om hvor mye bedre P2P skalerer ?



File distribution: client-server vs P2P

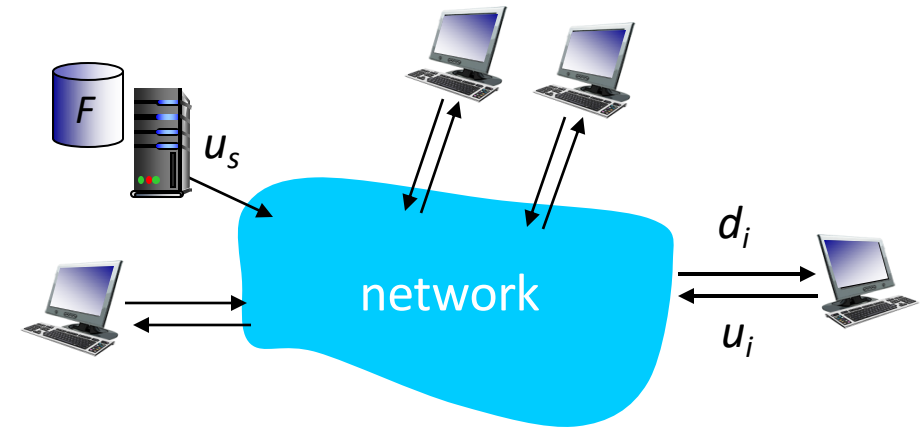
Q: how much time to distribute file (size F) from one server to N peers?

- peer upload/download capacity is limited resource



File distribution time: client-server

- **server transmission:** must sequentially send (upload) N file copies:
 - time to send one copy: F/u_s
 - time to send N copies: NF/u_s
- **client:** each client must download file copy
 - $d_{\min}$ = min client download rate
 - min client download time: $F/d_{\min}$

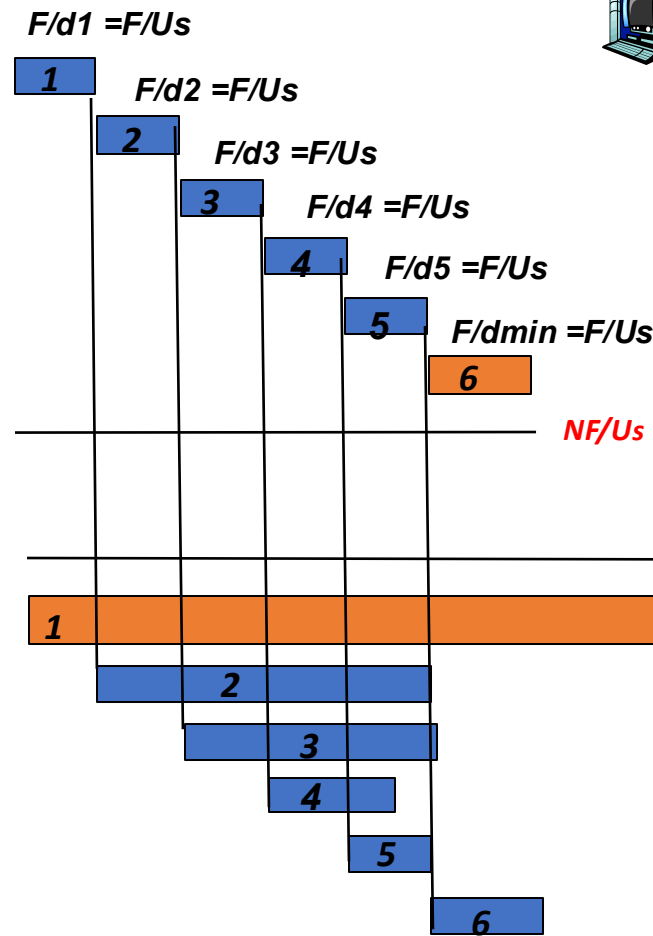
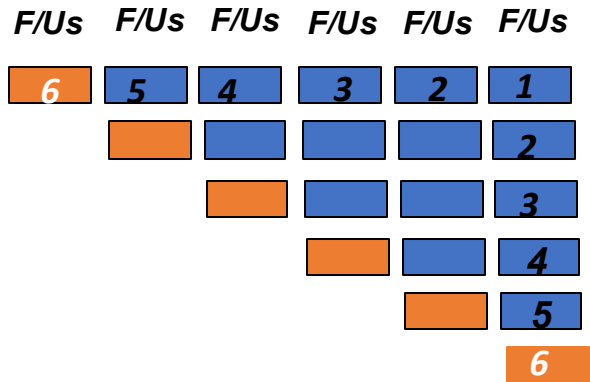
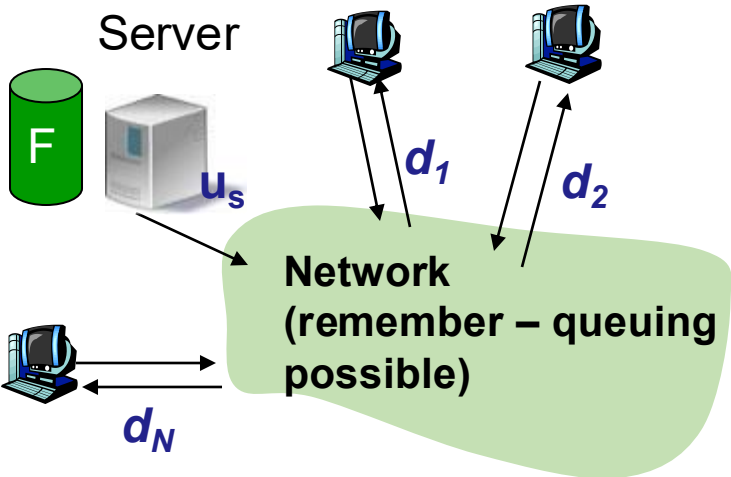


time to distribute F
to N clients using
client-server approach

$$D_{c-s} \geq \max\{ NF/u_s, F/d_{\min} \}$$

increases linearly in N

$$D_{C-S} \geq \max\{ NF/u_s, F/d_{\min} \}$$



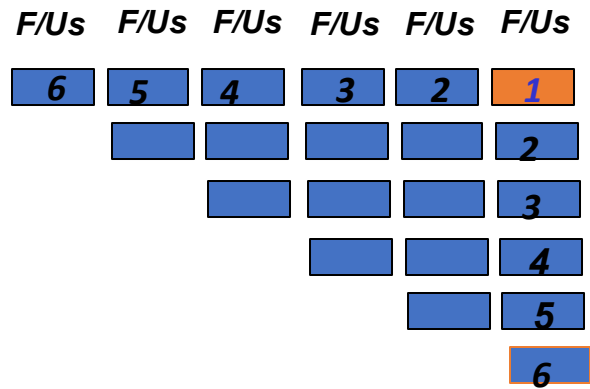
Selv om fysisk d_i er høyere enn u_s kommer ikke bits raskere fra server enn u_s , effektiv d_i blir derfor aldri høyere enn u_s

Her er $d_{\min} \geq u_s$, det går ikke raskere enn dette, og da er vi på NF/u_s

Vi kan se på dette som at pakke for pakke sendes med hastighet u_s

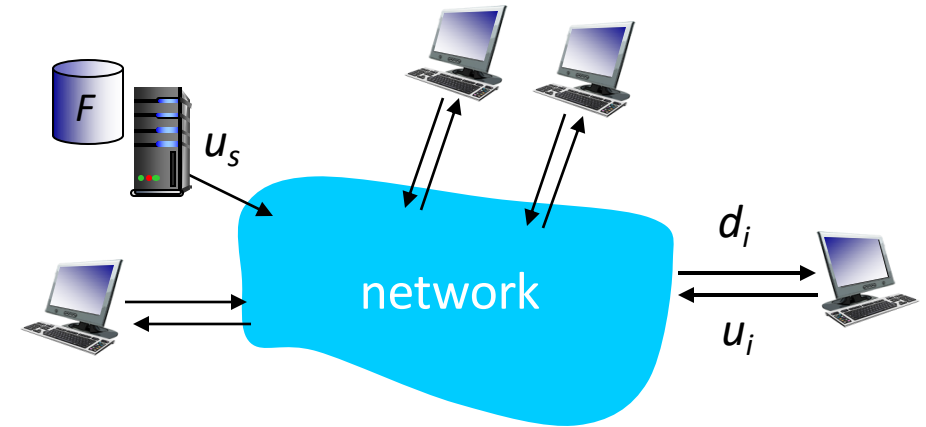
Her må vi se for oss et scenario hvor flere $d_i < u_s$ og pakkene ligger i kø undeveis i nettverket (selv om de sendes sekvensielt inn i nettet)

Den tregeste (lavest kapasitet) d_i vil da potensielt dominere og gi en tid for at alt skal bli ferdig som er større enn NF/u_s . Og la oss se for oss at grensetilfellet er når den tregeste starter først



File distribution time: P2P

- **server transmission:** must upload at least one copy:
 - time to send one copy: F/u_s
- **client:** each client must download file copy (vi må da se på den tregeste)
 - min client download time: F/d_{min}
- **clients:** as aggregate must download NF bits
 - max upload rate (limiting max download rate) is $u_s + \sum u_i$



time to distribute F
to N clients using
P2P approach

$$D_{P2P} \geq \max\{F/u_s, F/d_{min}, NF/(u_s + \sum u_i)\}$$

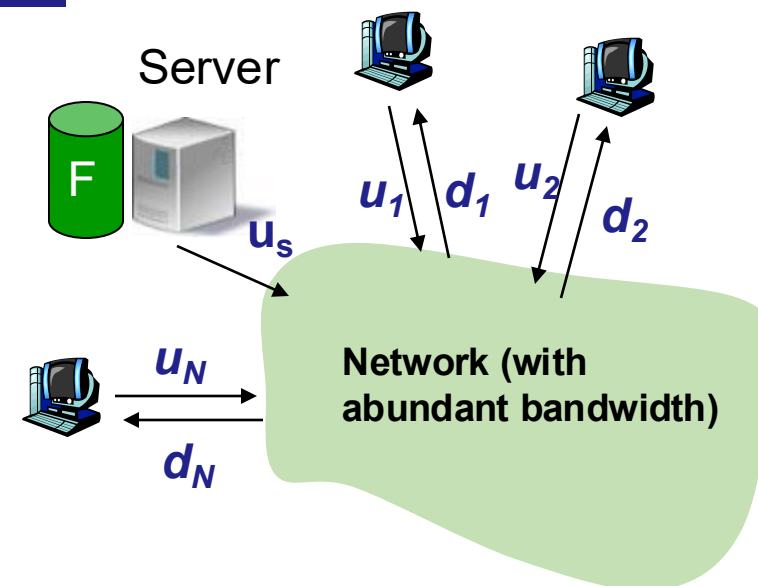
increases linearly in N ...
... but so does this, as each peer brings service capacity

I utgangspunktet så ligger jo filen på serveren, og hele denne må inn i nettet for at den skal ha mulighet til å komme seg videre til alle klientene. Det vil med andre ord si at dette er den absolutt minste tiden det kan ta å få distribuert en fil i P2P

Den ideelle situasjonen hvor filen som skal distribueres ligger sånn fordelt på alle peers (klienter og server) at summen av alle uplink kapasitetene kan benyttes til det fulle for å distribuere N kopier av F

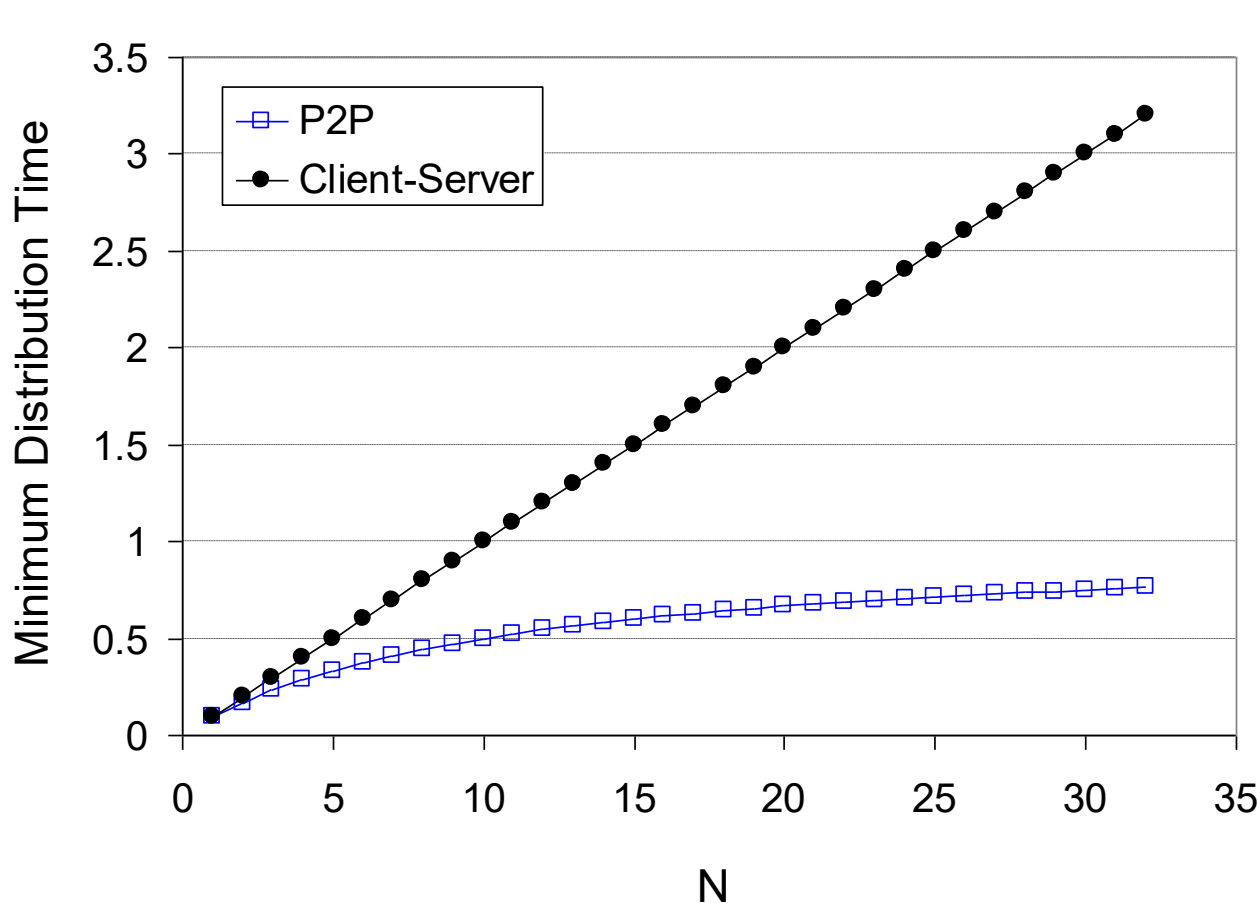
$$D_{P2P} \geq \max \{ F/u_s, F/d_{min}, NF/(u_s + \sum u_i) \}$$

Og på samme måte som for Client Server så vil den tregeeste klienten aldri klare å få fatt i filen raskere enn dette. Den begrenses av sin egen downlink kapasitet



Client-server vs. P2P: example (sammenligner minimum distrib tid)

Client **upload rate** = u , $F/u = 1$ hour, $u_s = 10u$ (altså 10 ganger client upload)
 Smallest **download rate** $d_{\min} \geq u_s$ (so no bottleneck)



$$\max \{ NF/u_s, F/d_{\min} \}$$

$$NF/u_s = NF/10u = N/10$$

$$\max \{ F/u_s, F/d_{\min}, NF/(u_s + \sum u_i) \}$$

$$NF/(u_s + \sum u_i) =$$

$$NF/(10u + Nu) =$$

$$NF/(10 + N)u =$$

$$(F/u)N/(10 + N) = N/(10 + N)$$

Application layer: overview

- Principles of network applications
- Web and HTTP
- E-mail, SMTP, IMAP
- The Domain Name System DNS
- P2P applications
- **video streaming and content distribution networks**
- socket programming with UDP and TCP



Video Streaming and CDNs: context

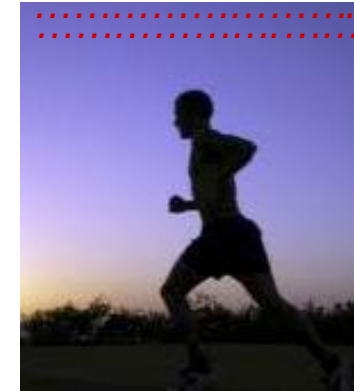
- stream video traffic: major consumer of Internet bandwidth
 - Netflix, YouTube, Amazon Prime: **80% of residential ISP traffic (2020)**
- challenge: scale - how to reach ~1B users?
 - single mega-video server won't work (why?)
- challenge: heterogeneity
 - different users have different capabilities (e.g., wired versus mobile; bandwidth rich versus bandwidth poor)
- ***solution: distributed, application-level infrastructure***



Multimedia: video

- video: sequence of images displayed at constant rate
 - e.g., 24 images/sec
- digital image: array of pixels
 - each pixel represented by bits
- coding: use redundancy *within* and *between* images to decrease # bits used to encode image
 - **spatial** (within image)
 - **temporal** (from one image to next)

spatial coding example: instead of sending N values of same color (all purple), send only two values: color value (*purple*) and number of repeated values (N)



frame i

temporal coding example: instead of sending complete frame at $i+1$, send only differences from frame i

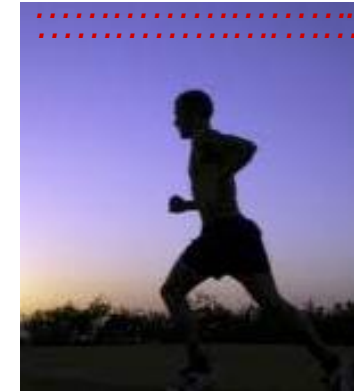


frame $i+1$

Multimedia: video

- **CBR: (constant bit rate):** video encoding rate fixed
- **VBR: (variable bit rate):** video encoding rate changes as amount of **spatial**, **temporal** coding changes
- **examples:**
 - MPEG 1 (CD-ROM) 1.5 Mbps
 - MPEG2 (DVD) 3-6 Mbps
 - MPEG4 (often used in Internet, 64Kbps – 12 Mbps)

spatial coding example: instead of sending N values of same color (all purple), send only two values: color value (*purple*) and number of repeated values (N)



frame i

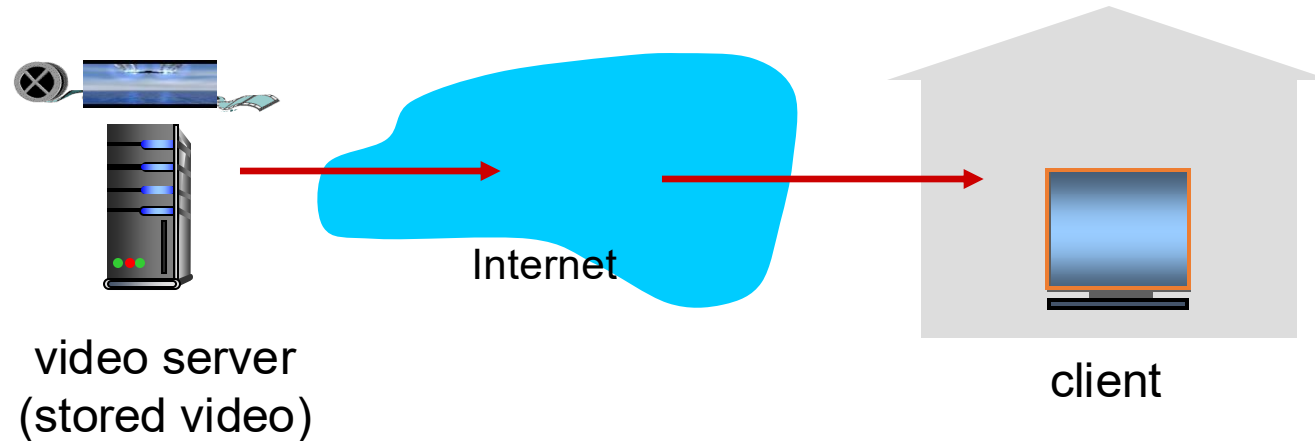
temporal coding example: instead of sending complete frame at $i+1$, send only differences from frame i



frame $i+1$

Streaming stored video

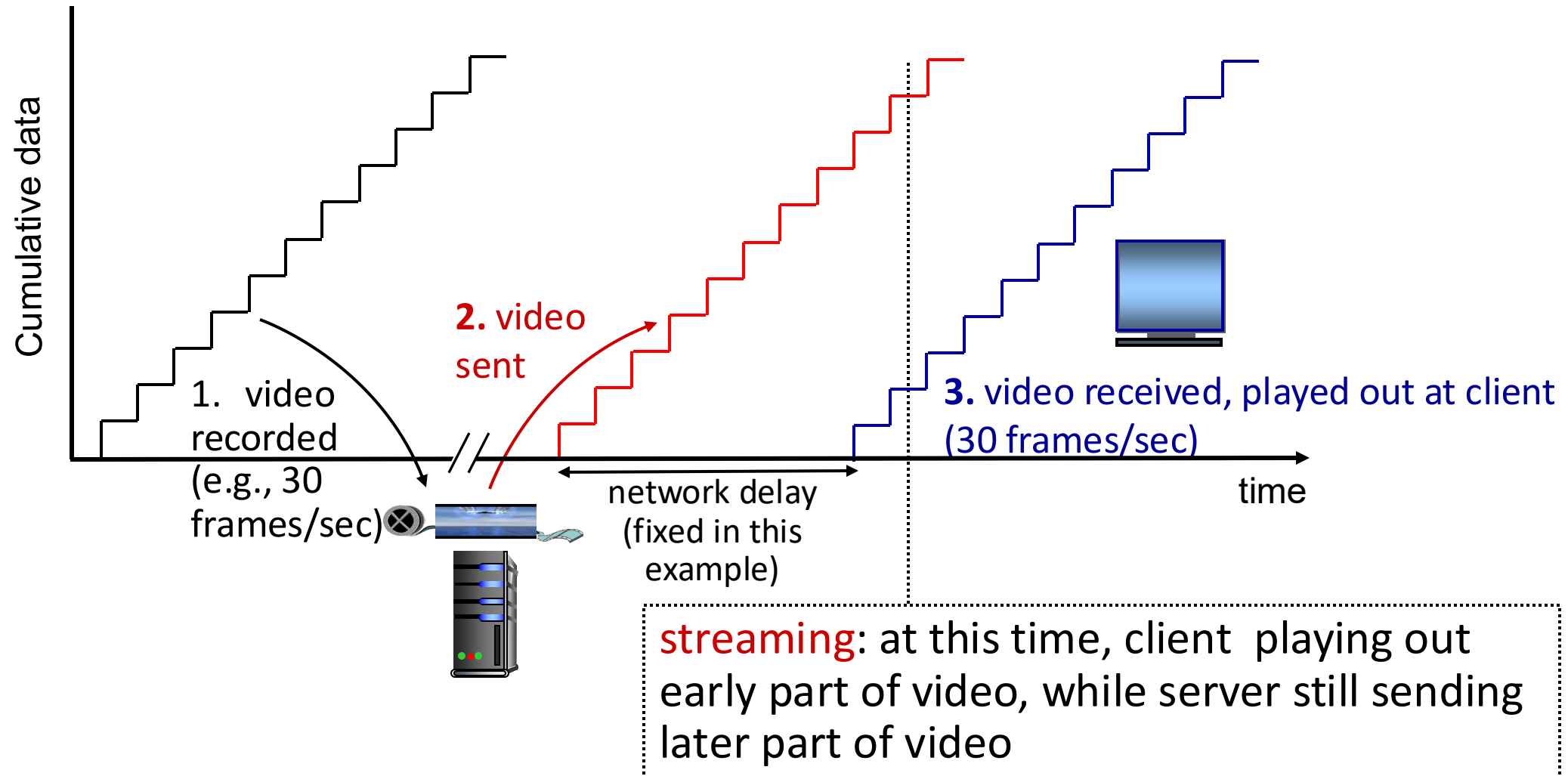
simple scenario:



Main challenges:

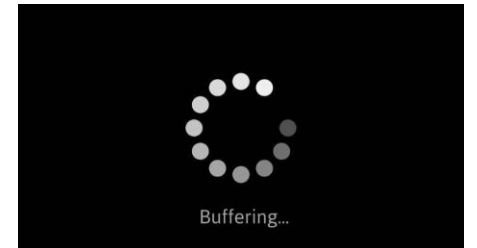
- server-to-client bandwidth will *vary* over time, with changing network congestion levels (in house, in access network, in network core, at video server)
- packet loss and delay due to congestion will delay playout, or result in poor video quality

Streaming stored video

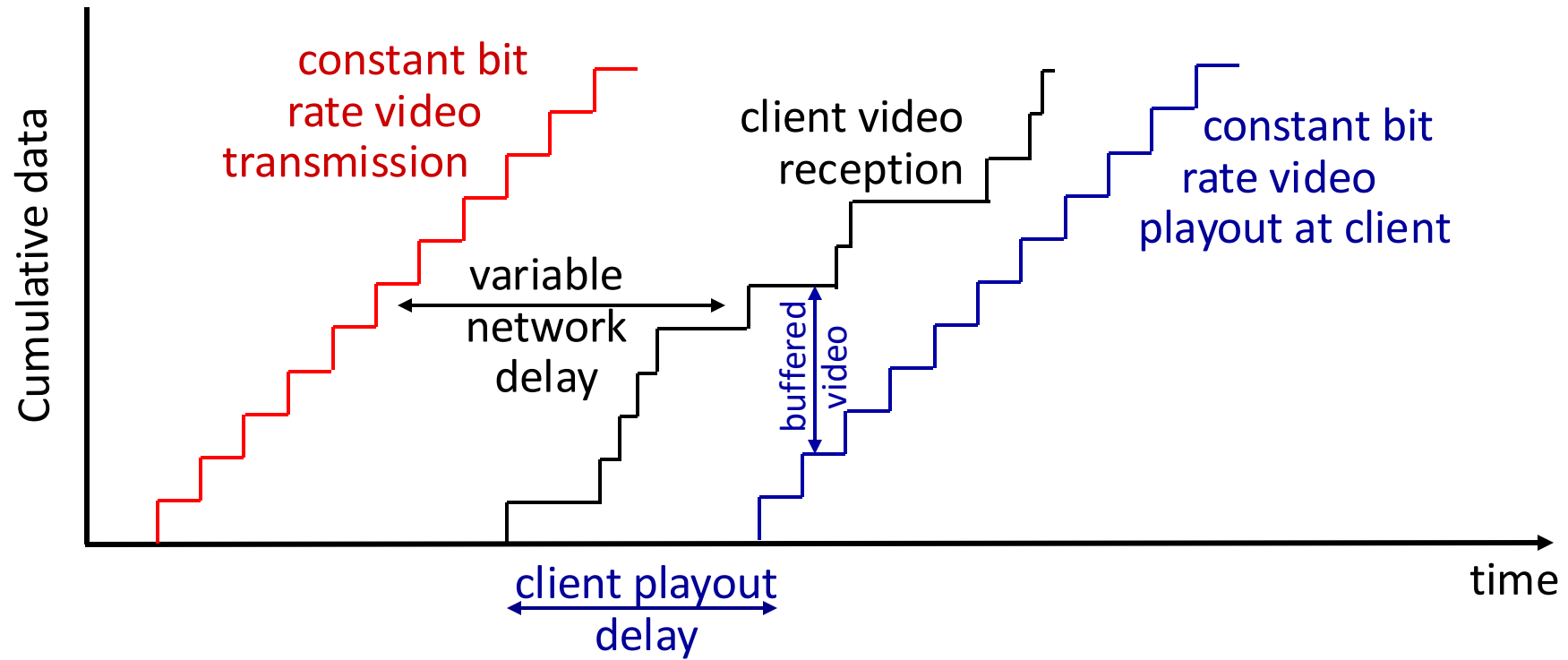


Streaming stored video: challenges

- **continuous playout constraint**: once client playout begins, playback must match original timing
 - ... but **network delays are variable** (jitter), so will need **client-side buffer** to match playout requirements
- other challenges:
 - client interactivity: pause, fast-forward, rewind, jump through video
 - video packets may be lost, retransmitted



Streaming stored video: playout buffering



- *client-side buffering and playout delay*: compensate for network-added delay, delay jitter

Streaming multimedia: DASH

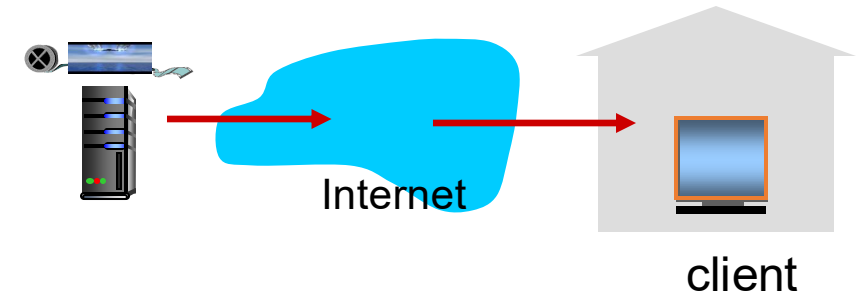
■ *DASH*: *D*ynamic, *A*daptive *S*treaming over *H*TTP

■ *server*:

- divides video file into multiple chunks
- each chunk stored, encoded at different rates
- *manifest file*: provides URLs for different chunks

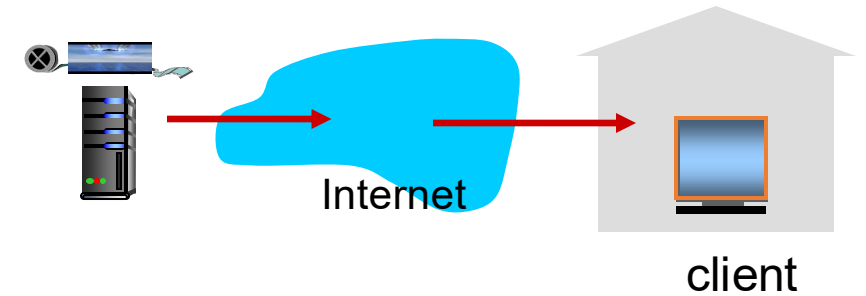
■ *client*:

- periodically measures server-to-client bandwidth
- consulting manifest, requests one chunk at a time
 - chooses maximum coding rate sustainable given current bandwidth
 - can choose different coding rates at different points in time (depending on available bandwidth at time)



Streaming multimedia: DASH

- “*intelligence*” at client: client determines
 - *when* to request chunk (so that buffer starvation, or overflow does not occur)
 - *what encoding rate* to request (higher quality when more bandwidth available)
 - *where* to request chunk (can request from URL server that is “close” to client or has high available bandwidth)



Streaming video = encoding + DASH + playout buffering

Content distribution networks (CDNs)

- *challenge*: how to stream content (selected from millions of videos) to hundreds of thousands of *simultaneous* users?
- *option 1*: single, large “mega-server”
 - single point of failure
 - point of network congestion
 - long path to distant clients
 - multiple copies of video sent over outgoing link

....quite simply: this solution *doesn't scale*

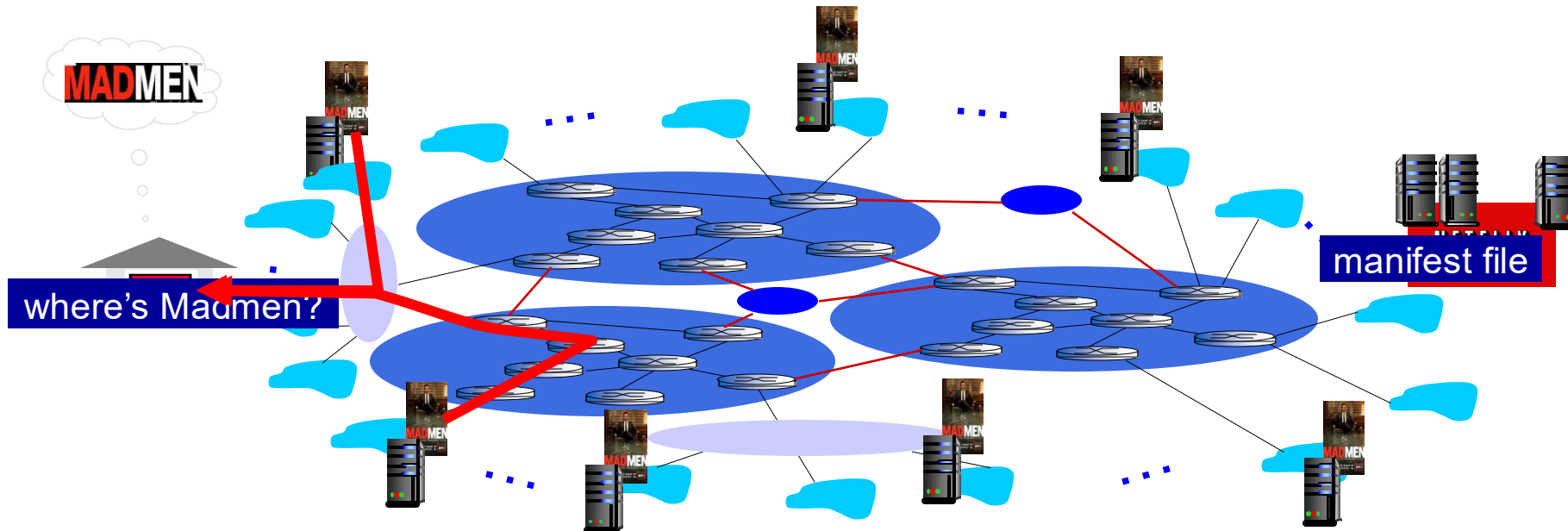
Content distribution networks (CDNs)

- *challenge*: how to stream content (selected from millions of videos) to hundreds of thousands of *simultaneous* users?
- *option 2*: store/serve multiple copies of videos at multiple geographically distributed sites (*CDN*)
 - *enter deep*: push CDN servers deep into many access networks
 - close to users
 - Akamai: 240,000 servers deployed in more than 120 countries (2015)
 - *bring home*: smaller number (10's) of larger clusters in POPs near (but not within) access networks
 - used by Limelight



Content distribution networks (CDNs)

- CDN: stores copies of content at CDN nodes
 - e.g. Netflix stores copies of MadMen
- subscriber requests content from CDN
 - directed to nearby copy, retrieves content
 - may choose different copy if network path congested



Content distribution networks (CDNs)



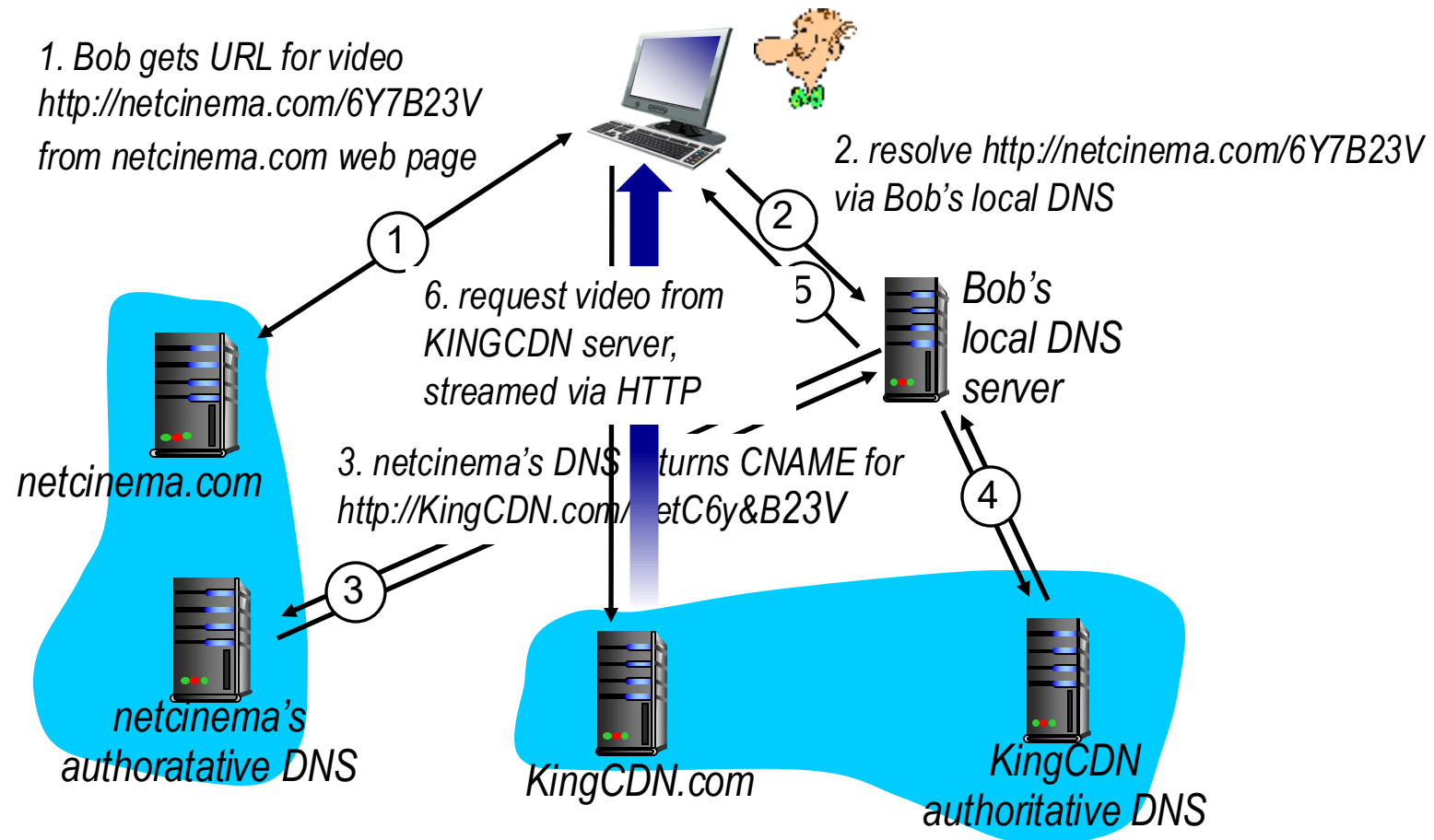
OTT challenges: coping with a congested Internet

- from which CDN node to retrieve content?
- viewer behavior in presence of congestion?
- what content to place in which CDN node?

CDN content access: a closer look

Bob (client) requests video `http://netcinema.com/6Y7B23V`

- video stored in CDN at `http://KingCDN.com/NetC6y&B23V`



Case study: Netflix

